# 1 Time Series Comparison and Prediction

For this assignment, we will switch to another dataset, namely the Kaggle Berkeley Earth Surface Temperature Study dataset[1]. We use the file `GlobalLandTemperaturesByCountry.csv`, which contains **monthly** average land temperatures per country up to September 2013. For the last question we additionally use hourly temperature data from the Open-Meteo historical weather archive[2].

Each question is marked with its level: BASIC, REGULAR or ADVANCED.

### (a) Data preparation. BASIC

Select the monthly temperatures of Norway, Finland, Singapore and Cambodia. The series start in different years and contain missing values. Choose a common period (e.g. 1900–2012), explain how you handle missing values, and plot both the monthly series and the yearly averages of the four countries.

*Hint:*

```
df = pd.read_csv('GlobalLandTemperaturesByCountry.csv', parse_dates=['dt'])
y = df[df.Country == 'Norway'].set_index('dt')['AverageTemperature']
y = y.loc['1900':'2012']
y.isna().sum();  y.interpolate();  y.resample('YS').mean()
```

### (b) Comparison with DTW. REGULAR

Compute the pairwise DTW distances between the yearly averages of the four countries, (i) on the raw data and (ii) after z-normalising each series. Also compute the Euclidean distance for (ii). Present the results as distance matrices and reflect:

- What does DTW measure in each case?
- All series share the same calendar years. Is warping in time meaningful here?
- What is the effect of a warping window (Sakoe–Chiba band)?

*Hint:* DTW is not part of `statsmodels`; use `tslearn`.

```
from tslearn.metrics import dtw
dtw(s1, s2)
dtw(s1, s2, global_constraint='sakoe_chiba', sakoe_chiba_radius=r)
z = (s - s.mean()) / s.std()          # z-normalisation
np.linalg.norm(z1 - z2)               # Euclidean distance
```

### (c) Stationarity. BASIC

Use the (augmented) Dickey–Fuller test to evaluate the stationarity of the monthly temperature series of the four countries. Repeat the test after first-order differencing and after seasonal differencing (lag 12). Interpret the results.

*Hint:*

```
from statsmodels.tsa.stattools import adfuller
stat, pvalue, *_ = adfuller(y)
y.diff().dropna();  y.diff(12).dropna()
```

[1] `https://www.kaggle.com/datasets/berkeleyearth/climate-change-earth-surface-temperature-data`
[2] `https://open-meteo.com/en/docs/historical-weather-api`

### (d) Trend and seasonality. REGULAR

Remove trend and seasonality from the monthly data using (i) differencing, (ii) classical decomposition and (iii) STL decomposition.

- Compare the seasonal components of classical decomposition and STL. Does the seasonal pattern change over time?
- Plot the trend components of the four countries in one figure and reflect on warming.
- Apply DTW again on the trend components and on the remainders, and compare with your results from (b).

*Hint:*

```
from statsmodels.tsa.seasonal import seasonal_decompose, STL
res_cl  = seasonal_decompose(y, model='additive', period=12)
res_stl = STL(y, period=12, robust=True).fit()
res_stl.trend;  res_stl.seasonal;  res_stl.resid;  res_stl.plot()
```

Note that classical decomposition gives `NaN` at the start and end of the trend.

### (e) Forecast comparison. REGULAR

Choose one country. Hold out the last 24 months as a test set and fit the following models on the remaining data:

- a *seasonal naïve* forecast (the value from the same month one year earlier) as a baseline;
- *Holt–Winters* exponential smoothing (additive or multiplicative season; motivate your choice);
- an *ARIMA* model, as you learned in the previous assignment. Choose $(p, d, q)$ from the ACF and PACF plots of the (differenced) training data and motivate your choice.

Forecast the 24 test months with each model, plot the forecasts together with the actual values, and report RMSE and MAE in a table. Which model is best, and does it beat the baseline? Reflect on why. In particular, how well does ARIMA capture the yearly cycle compared with Holt–Winters?

*Hint:*

```
train, test = y[:-24], y[-24:]

# seasonal naive
fc_naive = np.tile(train[-12:].values, 2)

# Holt-Winters
from statsmodels.tsa.holtwinters import ExponentialSmoothing
hw = ExponentialSmoothing(train, trend='add', seasonal='add',
                          seasonal_periods=12).fit()
fc_hw = hw.forecast(24)

# ARIMA: choose (p, d, q) from ACF / PACF
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
plot_acf(train.diff().dropna(), lags=36)
plot_pacf(train.diff().dropna(), lags=36)
from statsmodels.tsa.arima.model import ARIMA
ar = ARIMA(train, order=(p, d, q)).fit()
fc_ar = ar.forecast(24)
```

```
# errors
from sklearn.metrics import mean_squared_error, mean_absolute_error
rmse = np.sqrt(mean_squared_error(test, fc_hw))
```

### (f) Multiple seasonality. ADVANCED

Download two to three years of *hourly* temperature for the capital of your chosen country from the Open-Meteo archive (variable `temperature_2m`). For example, for Oslo:

```
https://archive-api.open-meteo.com/v1/archive?latitude=59.91&longitude=10.75&
    start_date=2022-01-01&end_date=2024-12-31&hourly=temperature_2m&format=csv
```

1. Identify the seasonal periods in the data from the plot, the ACF and/or the periodogram. Which periods do you find, expressed in hours?
2. Decompose the series with MSTL using these periods, and compare with a single STL using only the daily period. What goes wrong with only one period?
3. Hold out the last seven days and forecast them with (i) a seasonal naïve forecast (the same hour one day earlier) and (ii) a model that handles multiple seasonality: remove the seasonal components found by MSTL, forecast the seasonally adjusted series with ETS or ARIMA, and add the seasonal components back. Compare the forecasts with the actual values.
4. Electricity demand typically shows a *weekly* cycle in addition to daily and yearly cycles. Why is there no weekly cycle in temperature data?

*Hint:* The CSV starts with a few metadata lines; inspect the file first.

```
y = pd.read_csv(url, skiprows=3, parse_dates=['time'],
                index_col='time').iloc[:, 0]

# ACF and periodogram
plot_acf(y, lags=24*10)
from scipy.signal import periodogram
f, p = periodogram(y)                  # period in hours = 1 / f

# MSTL (statsmodels >= 0.14) and single STL
from statsmodels.tsa.seasonal import MSTL, STL
res = MSTL(y, periods=(24, 24*365)).fit()   # columns of res.seasonal
res1 = STL(y, period=24).fit()

# seasonally adjusted series -> forecast -> add seasonality back
y_adj = y - res.seasonal.sum(axis=1)
ExponentialSmoothing(y_adj, trend='add').fit().forecast(24*7)
```

For a single seasonal period, `statsmodels` offers this in one step: `from statsmodels.tsa.forecasting.stl import STLForecast`.